Sensitivity Labeling for Power BI & Microsoft Fabric — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Sensitivity Labeling for Power BI & Microsoft Fabric: Microsoft Purview 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview) 5

What this solution is . . . 5

Why this matters . . . 6

How it works . . . 6

The analytics labeling design (used throughout) . . . 6

What you get . . . 7

2. Requirements Guide — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview) 7

2.1 Sensitivity-label entitlement plus Power BI Pro/PPU (or a capacity) . . . 8

2.2 Least-privilege roles across Purview and Power BI/Fabric . . . 8

2.3 Security & Compliance PowerShell + read-only Power BI toolchain . . . 8

2.4 Information-protection tenant settings available in the Power BI/Fabric admin portal . . . 8

2.5 A delegation security group and in-scope workspaces . . . 9

2.6 Published sensitivity labels compatible with analytics . . . 9

2.7 Labeled data sources for inheritance (where inheritance is in scope) . . . 9

2.8 In-scope content, classification intent, stakeholders, and a pilot ring . . . 9

Checklist . . . 10

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 10

2. Purview roles (least-privilege) and portal access . . . 11

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 11

5. Compliance, records, privacy & legal stakeholders . . . 11

6. A pilot scope and change control . . . 12

3. Implementation Guide — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview) 12

Step 0 — Connect (all steps) . . . 13

Step 1 — Assess current state (Assess) . . . 13

Step 2 — Design the analytics labeling (Design) . . . 13

Step 3 — Publish the labels and enable the tenant settings (Configure) . . . 13

Step 4 — Pilot on real content (Pilot) . . . 14
Step 5 — Validate the pilot (Pilot → Validate) . . . 14
Step 6 — Expand to the estate (Expand) . . . 14
5. Testing Guide — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview) 15
Test 1 — Prerequisites and enablement are in place . . . 15
Test 2 — Discovery inventory and tenant-setting checklist are complete . . . 15
Test 3 — Publish dry-run makes no changes . . . 16
Test 4 — Labels are published and appear in the service . . . 16
Test 5 — Tenant settings are enabled for the pilot group . . . 16
Test 6 — Validation gate passes on the pilot . . . 16
Test 7 — An artifact can be labeled . . . 16
Test 8 — The label inherits from a labeled source . . . 16
Test 9 — The label inherits to an export . . . 16
Test 10 — Export restriction behaves as designed . . . 17
Test 11 — Reporting evidence is produced . . . 17
6. Troubleshooting — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview) 17
The sensitivity-label picker doesn't appear in the Power BI service . . . 17
The label doesn't inherit from a labeled data source . . . 17
An export to Excel/PowerPoint/PDF isn't protected . . . 18
Mandatory labeling is blocking creators unexpectedly . . . 18
`IPPBI-Set-InfoProtection.ps1 -Commit` fails to publish the policy . . . 18
`IPPBI-Discover.ps1` can't read workspaces . . . 18
`IPPBI-Validate.ps1` says a label is "not detected on the policy" but the portal shows it . . . 19
7. Best Practices — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview) 19
8. Deliverables Summary — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview) 20
Documentation . . . 20
Named artifacts . . . 21
PowerShell scripts . . . 21
What the buyer receives . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Sensitivity Labeling for Power BI & Microsoft Fabric: Microsoft Purview

Analytics is where classified data is at its most concentrated — and its most portable. A Power BI report or a Microsoft Fabric item can join HR, finance, and customer data in one place, and then a user clicks **Export to Excel** and the data leaves in a spreadsheet that carries no protection at all. You cannot claim your analytics estate is governed when a download strips the classification. This solution closes that gap: it applies **Microsoft Purview sensitivity labels** to Power BI/Fabric content and makes those labels **flow with the data** — inherited from labeled sources, carried into exports with their encryption — configured, piloted, validated, and defensible.

This is a complete, do-it-today package written for an administrator enabling analytics labeling for the first time. It reuses your existing Purview sensitivity labels — not a BI-only set — and delivers the capability end-to-end: from reading the current labels and the Power BI/Fabric information-protection tenant settings, through publishing labels to creators and enabling labeling/inheritance/export restriction, to piloting on a real workspace and expanding — with every label, policy, group, and pilot scope named once and reused everywhere, and every change proven on a pilot before the estate is touched. Because the analytics information-protection surface is **evolving**, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a setting or entitlement as permanent.

What you get

• A plain-language Solution Overview of analytics labeling — artifact labeling, source inheritance, and export inheritance — and how Power BI/Fabric honor your existing Purview labels.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: the sensitivity-label entitlement plus Power BI Pro/PPU (or a capacity), least-privilege Purview and Power BI/Fabric roles, the S&C PowerShell + Power BI toolchain, the information-protection tenant settings, a delegation group and pilot workspace, published labels, labeled sources for inheritance, and stakeholders with change control.

• An Implementation Guide of precise, ordered steps: assess labels and tenant settings, design the intent and map, publish labels to a pilot, enable the tenant settings (labeling + source + export inheritance), **pilot on real content**, validate against the gate, then expand with mandatory labeling.

• Five PowerShell tools — Discovery, a prerequisite/health helper, a safe Configuration/Remediation tool (`-WhatIf` by default), Validation, and Reporting — built on Security & Compliance PowerShell and read-only Power BI PowerShell, **read-only or `-WhatIf` by default with no credentials in code**.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

Analytics content and its exports protected from Purview: Power BI/Fabric artifacts labeled, labels inherited from labeled sources and carried into Excel/PowerPoint/PDF exports with their encryption — validated on a pilot before you scale.

The deployment model

Assess → Design → Configure → **Pilot (real workspace)** → Validate → Expand — with a pilot gate before the estate, and mandatory labeling turned on only after validation.

Outcome

The analytics export blind spot closed — a labeled report produces a labeled, still-protected export — with the determination of which content is sensitive, and any regulatory/records obligation on the data it surfaces, left to the owners who make it.

1. Solution Overview — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview)

What this solution is

Analytics is where classified data is at its most concentrated and its most portable. A Power BI report or a Microsoft Fabric item can surface data from a dozen sources at once — and then a user clicks **Export to Excel**, and the data walks out the door in a spreadsheet that carries no protection at all. That is the gap this solution closes. It applies **Microsoft Purview sensitivity labels** to Power BI and Fabric content and, critically, makes those labels **flow with the data**:

• **Label the analytics artifact** — apply a sensitivity label to a Power BI **semantic model, report, dashboard, dataflow**, and to** Microsoft Fabric items** (lakehouses, warehouses, notebooks, etc.), so the content itself is classified in the Purview taxonomy.

• **Inherit from the source** — where a data source is already labeled (for example a labeled file or a labeled database column), the label can **flow into** the Power BI content, so classification isn't re-decided in the BI layer.

• **Inherit to the export** — when a user exports to **Excel, PowerPoint, or PDF**, the sensitivity label and its **encryption/usage rights travel with the exported file**, so a "download" is no longer a way to strip protection.

• **Drive it with a policy** — publish the labels to BI creators, and (after piloting) require a label on new/changed content with **mandatory labeling**, so coverage doesn't depend on memory.

Everything is done with the **Microsoft Purview portal** (purview.microsoft.com), the **Power BI / Fabric admin portal**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and read-only Power BI PowerShell — no third-party tooling. The capability is enabled, **piloted on a small ring/workspace first**, validated against pass/fail gates, then expanded.

> ***The labels are your existing Purview sensitivity labels — this solution makes analytics honor them.*** *Power BI and Fabric do not use a separate label set; they consume the same* ***Information Protection*** *sensitivity labels published from Purview. This solution enables analytics to apply and inherit those labels and enforce them on export — it does not invent BI-only labels.*

Why this matters

• **Export is the real exfiltration path.** The dashboard sits behind Power BI's access controls, but the exported spreadsheet doesn't — unless the label and its encryption ride along. Export inheritance turns every download into a still-protected file.

• **Analytics concentrates sensitivity.** One report can join HR, finance, and customer data. Labeling the artifact makes that concentration visible and governed instead of implicit.

• **Inheritance prevents re-classification drift.** If a labeled source flows into an unlabeled report, the protection is lost at the BI boundary. Source-to-content inheritance keeps the classification consistent from the data to the visual to the export.

• **It is defensible and auditable.** "Every workspace containing regulated data enforces labels, and exports inherit encryption" is a statement you can evidence with a report and admin settings, not a hope.

How it works

1. **Assess.** Read the tenant's current state — which sensitivity labels exist and are published, the **Power BI/Fabric tenant settings** for information protection (label application, source inheritance, mandatory labeling, export restrictions), and the in-scope workspaces — all read-only.

1. **Design.** Decide which sensitivity labels apply to analytics, which **security group** may apply labels, whether **source inheritance** and **export restrictions** are on, and where **mandatory labeling** will eventually apply — aligned to the classification intent your stakeholders set.

1. **Configure.** Publish the labels to BI creators via a label policy, and enable the information-protection **tenant settings** in the Power BI/Fabric admin portal (delegated to the security group).

1. **Pilot.** Enable for a **pilot workspace / security group first** and label real content — never tenant-wide on day one. Mandatory labeling and export restrictions are **high-impact** for creators and consumers; prove them on the ring.

1. **Validate.** Confirm labels apply to artifacts, **inherit from labeled sources**, and **inherit to exports** (an exported Excel carries the label and its protection), against the pass/fail gates.

1. **Expand.** Broaden to all intended workspaces, enable **mandatory labeling** only after validation, and hand ongoing coverage and reporting to operations.

The analytics labeling design (used throughout)

The same organizational sensitivity labels are reused; this solution governs **how analytics applies and inherits them**. Names below are examples — the **determination** of which content warrants which label belongs to your compliance, security, and data-governance stakeholders.

Setting	Design intent
Labels published to BI creators	`General`, `Confidential`, `Highly Confidential` (your existing Purview labels) via `Analytics Labels – Policy`
Who may apply labels	`BI Labeling – Enabled Users` security group (tenant-setting delegation)
Source → content inheritance	**On** — labeled sources flow their label into Power BI content
Export → file inheritance	**On** — Excel/PowerPoint/PDF exports inherit the label + encryption
Mandatory labeling	**Off during pilot; on after validation** — new/changed content must be labeled

> ***Licensing / plan note — verify against your own tenant.*** *Applying sensitivity labels in Power BI requires **Power BI Pro or Premium Per User (or a Premium/Fabric capacity)** for the affected users **and** the sensitivity-label entitlement (Azure Information Protection P1/P2, included in Microsoft 365 E5 / E5 Compliance). Information-protection **tenant settings** are configured in the Power BI/Fabric admin portal and are **evolving** — confirm the current setting names, the export-restriction behavior, and Fabric-item coverage for *your* tenant in Microsoft's documentation before you design. Where a capability is unavailable, scope to what your plan supports and record the rest as **deferred**.*

What you get

• This **Solution Overview**, a **Requirements Guide** (every prerequisite with verify / implement-if-missing), a precise **Implementation Guide**, a **Testing Guide** with pass/fail gates, a **Troubleshooting Guide**, and field-proven **Best Practices**.

• **Five PowerShell tools** — Discovery, a prerequisite/health helper, a safe Configuration/Remediation tool (`-WhatIf` by default), Validation, and Reporting — built on Security & Compliance PowerShell and read-only Power BI PowerShell, **read-only or `-WhatIf` by default and with no credentials in code**.

> ***Boundary.*** *Which analytics content is sensitive, and any regulatory/records obligation on the data it surfaces, are **determinations owned by your compliance, privacy, and data-governance functions**. This solution provides the technical configuration and the evidence — not the determination.*

2. Requirements Guide — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-purview.md` (the six shared requirements: compliance licensing, least-privilege Purview roles, the S&C PowerShell + Graph toolchain, in-scope data sources, compliance/records/privacy/legal stakeholders, and a pilot scope with change control) — those are referenced here, not repeated.

2.1 Sensitivity-label entitlement plus Power BI Pro/PPU (or a capacity)

Description: A tenant licensed for **sensitivity labels** (Azure Information Protection P1/P2 — included in Microsoft 365 E5 / E5 Compliance) **and** the users who create/consume labeled Power BI content licensed for **Power BI Pro or Premium Per User**, or working in a **Premium/Fabric capacity**. (Global Requirement 1.) **Why:** Applying and enforcing sensitivity labels in Power BI/Fabric is gated on both the label entitlement and the Power BI/Fabric licensing of the affected users. **Verify:** M365 admin center → `Billing → Licenses` shows the AIP/label SKU and Power BI Pro/PPU (or a capacity); `IPPBI-Check-Prereqs.ps1` reports the label and Power BI signals. **Verify against your own plan. Implement:** Assign the label entitlement and Power BI Pro/PPU (or assign the workspace to a capacity) for the in-scope creators/consumers before enabling. Where unavailable, record affected capabilities as **deferred**.

2.2 Least-privilege roles across Purview and Power BI/Fabric

Description: Information Protection Admin (or **Compliance Administrator**) to author/publish sensitivity labels and label policies in Purview; a **Power BI administrator / Fabric administrator** to configure the information-protection **tenant settings** in the Power BI/Fabric admin portal. Reserve **Global Administrator** for one-time enablement only. (Global Requirement 2 — least privilege.) **Why:** Labeling spans two surfaces — the label (Purview) and the analytics tenant settings (Power BI/Fabric admin portal) — each with its own least-privilege role. **Verify:** Purview portal → `Settings → Roles & scopes → Permissions` shows Information Protection Admin; Power BI/Fabric admin portal access confirmed; `Connect-IPPSSession` succeeds. **Implement:** Assign the least-privilege role on each surface under change control; do not use Global Administrator for day-to-day label authoring.

2.3 Security & Compliance PowerShell + read-only Power BI toolchain

Description: A workstation with **ExchangeOnlineManagement** (provides `Connect-IPPSSession` for `Get-Label` / `Get-LabelPolicy`) and, for read-only workspace inventory, the **MicrosoftPowerBIMgmt** module (`Connect-PowerBIServiceAccount` / `Get-PowerBIWorkspace`), with a modern-auth / MFA admin. (Global Requirement 3.) **Why:** Labels/policies are read/authored through Security & Compliance PowerShell; workspace inventory is read through the Power BI module; the provided scripts use both read-only where possible. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement, MicrosoftPowerBIMgmt`; the connect cmdlets succeed. `IPPBI-Check-Prereqs.ps1` reports which modules/connections are present. **Implement:** `Install-Module ExchangeOnlineManagement, MicrosoftPowerBIMgmt -Scope CurrentUser`; confirm modern-auth sign-in before running steps.

2.4 Information-protection tenant settings available in the Power BI/Fabric admin portal

Description: The **information protection** section of the Power BI/Fabric **admin portal** → **Tenant settings** must be available so you can enable **"Allow users to apply sensitivity labels for content"**, **"Apply sensitivity labels from data sources to their data"** (source inheritance), **restrict content that has protected/encrypted labels** (export behavior), and (later) mandatory labeling. **Why:** These tenant settings — not a Purview policy — are what turn labeling, inheritance, and export protection on for analytics. **Verify:** In the Power BI/Fabric admin portal, open **Tenant settings** → **Information protection** and confirm the settings are present and can be delegated to a security group. **Verify the exact setting**

names against current Microsoft guidance for your tenant — the analytics information-protection surface is evolving. **Implement:** Confirm availability for your licensing/tenant; complete any prerequisite Microsoft documents (for example, that sensitivity labels are published and that AIP is enabled) before enabling.

2.5 A delegation security group and in-scope workspaces

Description: A **security group** (`BI Labeling – Enabled Users`) to which the information-protection tenant settings are scoped, and the **in-scope Power BI/Fabric workspaces** where labeling applies — starting with a **pilot workspace/group** before the estate. **Why:** Tenant settings are best delegated to a group so labeling is enabled for the pilot first, not tenant-wide on day one. **Verify:** The security group exists and the intended pilot users/workspaces are members; `IPPBI-Discover.ps1` reports the in-scope workspaces. **Implement:** Create/populate the security group and confirm the pilot workspaces before enabling the tenant settings.

2.6 Published sensitivity labels compatible with analytics

Description: The organizational sensitivity labels (`General`, `Confidential`, `Highly Confidential` or your scheme) authored and **published** via a label policy (`Analytics Labels – Policy`) to the BI creators, so they are available to apply in Power BI/Fabric. **Why:** Power BI/Fabric consume published Purview labels; a label a creator can't see in the service can't be applied. **Verify:** `Get-Label` lists the labels; `Get-LabelPolicy` shows the policy publishing them to the creators; the label picker appears in the Power BI service for a policy member. `IPPBI-Check-Prereqs.ps1` confirms labels are present/published. **Implement:** Publish the labels to the BI creators (Implementation Guide) before enabling analytics labeling; pilot group first.

2.7 Labeled data sources for inheritance (where inheritance is in scope)

Description: For **source** → **content inheritance**, the upstream data sources whose labels should flow into Power BI (for example labeled files, or database columns with sensitivity metadata) must actually carry labels/classification. **Why:** Inheritance only flows a label that exists on the source; unlabeled sources produce unlabeled content regardless of the setting. **Verify:** Confirm the in-scope sources are labeled/classified; `IPPBI-Discover.ps1` reports the tenant setting state so you know inheritance is enabled. **Implement:** Ensure the upstream sources are labeled (a separate labeling effort) where inheritance is required; where sources are unlabeled, rely on artifact labeling and record inheritance as **partial**.

2.8 In-scope content, classification intent, stakeholders, and a pilot ring

Description: The **analytics content in scope**, the **classification intent** mapping content to labels, engagement with **compliance, privacy, security, and data-governance** stakeholders (Global Requirements 4 & 5), and a **pilot workspace/group** (`Analytics Labels – Pilot Ring – Stage1`) with change control (Global Requirement 6). **Why:** Which content gets which label is a business/classification decision; mandatory labeling and export restrictions are high-impact and must be piloted before the estate. **Verify:** In-scope content and the label mapping are agreed; named approvers recorded; the pilot workspace/group and a change window exist. **Implement:** Capture the classification intent and the pilot scope; agree change control before enabling mandatory labeling or expanding beyond the pilot.

Checklist

- Sensitivity-label entitlement (AIP/E5) + Power BI Pro/PPU (or capacity) confirmed for in-scope users
- Information Protection Admin (Purview) + Power BI/Fabric Administrator assigned least-privilege
- ExchangeOnlineManagement + MicrosoftPowerBIMgmt installed; `Connect-IPPSSession` succeeds
- Information-protection **tenant settings** available in the Power BI/Fabric admin portal (verified for your tenant)
- `BI Labeling – Enabled Users` security group + in-scope/pilot workspaces defined
- Sensitivity labels authored and `Analytics Labels – Policy` published to BI creators
- Labeled data sources confirmed where source inheritance is in scope (else recorded partial)
- In-scope content + label mapping agreed; stakeholders named; `Analytics Labels – Pilot Ring – Stage1` + change control

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

- **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

- **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

- **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

- **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

- **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.
- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

• **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.

• **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

• **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview)

Follow these steps in order. The model is **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**. Every write is **piloted on** `Analytics Labels – Pilot Ring – Stage1` **/ a pilot workspace first**, previewed with `-WhatIf`, and reversible. Named objects are introduced once and reused everywhere:

Object	Name
Sensitivity labels (existing)	`General`, `Confidential`, `Highly Confidential`
Label policy publishing to BI creators	`Analytics Labels – Policy`
Tenant-setting delegation group	`BI Labeling – Enabled Users`
Pilot ring (users) / pilot workspace	`Analytics Labels – Pilot Ring – Stage1`
Scope/design map	`Analytics Scope – Map`
Coverage report	`Analytics Labels – Coverage Report`

> ***Two surfaces, two jobs.*** *The **label lifecycle** (author/publish the sensitivity labels) is done in Purview and is scriptable.* ***Turning labeling, source inheritance, export restriction, and mandatory labeling ON for analytics*** *is done in the **Power BI/Fabric admin portal → Tenant settings → Information***

> ***protection**, delegated to `BI Labeling – Enabled Users`. Use the scripts for the safe label-policy lifecycle and for validation/reporting; complete the tenant settings in the admin portal. **Verify each setting's availability and name against your licensing/tenant.***

Step 0 — Connect (all steps)

Open PowerShell 5.1+ or 7+ and connect with a modern-auth/MFA admin:

```
Connect-IPPSSession                # Security & Compliance PowerShell (labels + policies)
Connect-PowerBIServiceAccount      # Power BI (read-only workspace inventory here)
```

Keep credentials out of scripts — sign in interactively. **Pass/fail:** both connections succeed and `Get-Label` / `Get-PowerBIWorkspace` return without error.

Step 1 — Assess current state (Assess)

1. Run `IPPBI-Discover.ps1`. Read-only, it reports: every sensitivity label and the label policies

publishing them; the in-scope **Power BI/Fabric workspaces**; and a checklist of the information-protection **tenant settings** to confirm in the admin portal (the script cannot read all tenant settings, so it prints them to verify).

1. Run `IPPBI-Check-Prereqs.ps1` — modules/connections, roles, the label and Power BI licensing signals,

and whether labels are published — PASS/GAP per signal.

1. Record every GAP against the Requirements checklist and remediate before configuring.

Pass/fail: the discovery inventory and prereq report complete; the tenant-setting checklist is captured in `Analytics Scope – Map`.

Step 2 — Design the analytics labeling (Design)

1. With your stakeholders, confirm which sensitivity labels apply to analytics, whether **source

inheritance **and** export restriction **are on, and where** mandatory labeling** will eventually apply.

1. Define `BI Labeling – Enabled Users` and the **pilot workspace/group**; map in-scope workspaces to the

intent. Record everything in `Analytics Scope – Map`.

1. Confirm which capabilities your licensing supports (Pro/PPU/capacity, AIP entitlement, Fabric-item

coverage); mark any unsupported control **deferred**. **Pass/fail:** `Analytics Scope – Map` is agreed and signed off; which content is which label is owned by stakeholders, not by IT.

Step 3 — Publish the labels and enable the tenant settings (Configure)

Do this **for the pilot first** — the delegation group and pilot workspace only.

1. **Publish labels to BI creators (scriptable).** Point `Analytics Labels – Policy` at the pilot

creators:

```
.\IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -PolicyMembers "BI-Labeling-Pilot@contoso.com" -Wh
```

```
.\IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -PolicyMembers "BI-Labeling-Pilot@contoso.com" -Co
```

Confirm the labels appear in the Power BI service label picker for a policy member.

1. **Enable the tenant settings (admin portal).** Power BI/Fabric **admin portal → Tenant settings → Information protection**. Scope each to `BI Labeling – Enabled Users`:

- **Allow users to apply sensitivity labels for content** — **On**.
- **Apply sensitivity labels from data sources to their data in Power BI** (source inheritance) — **On** (where inheritance is in scope).
- **Restrict content with protected labels from being accessed by unsupported apps/exports** and the related **export** protections — configure per design so exports inherit the label + encryption.
- **Mandatory label policy for Power BI** — **leave Off during the pilot**.

Use `IPPBI-Set-InfoProtection.ps1 -Action ShowTenantSettings` to print the exact checklist to apply. **Pass/fail:** `Get-Label` shows the labels; `Get-LabelPolicy` shows `Analytics Labels – Policy` scoped to the pilot creators; the label picker appears in the service for a pilot member and the tenant settings are enabled for `BI Labeling – Enabled Users`.

Step 4 — Pilot on real content (Pilot)

1. In the pilot workspace, have creators apply each label to real (or representative) semantic models, reports, and Fabric items.
1. Where a labeled source feeds a report, confirm the label **inherits** into the content. Then **export to Excel **and confirm the exported file carries the label and its protection.** Pass/fail:** labels apply to artifacts, inherit from labeled sources, and inherit to exports; discrepancies are recorded for troubleshooting, not worked around.

Step 5 — Validate the pilot (Pilot → Validate)

1. Run `IPPBI-Validate.ps1`. Read-only, it confirms the labels exist and are published in `Analytics Labels – Policy` to the pilot creators, and prints the tenant-setting gates to confirm in the admin portal — PASS/FAIL per gate.
1. Have a pilot creator confirm real behavior end-to-end: label an artifact, inherit from a labeled source, export to Excel and open the protected file.
1. Complete the **Testing Guide** on the pilot. **Pass/fail:** all Testing-Guide gates pass on `Analytics Labels – Pilot Ring – Stage1`.

Step 6 — Expand to the estate (Expand)

1. Broaden `Analytics Labels – Policy` to all intended BI creators and add the remaining workspaces/users

to `BI Labeling – Enabled Users` (or set the tenant settings to the whole organization once validated).

1. Enable **mandatory label policy for Power BI** only after the pilot validates and stakeholders approve; communicate the change to creators first.

1. Run `IPPBI-Report.ps1 -CsvPath .\AnalyticsLabelsReport.csv` to produce `Analytics Labels – Coverage Report` — the labels, their publication/target coverage, and the in-scope workspaces — as the management evidence artifact, and re-run it on a schedule. **Pass/fail:** labels are published to all intended creators; the tenant settings are enabled at the intended scope; the report shows the taxonomy present and targeted as designed.

Rollback

• **Un-publish** by removing a label from `Analytics Labels – Policy` (portal, or `Set-LabelPolicy ... -RemoveLabel`) or narrowing the policy back to the pilot creators.

• **Disable a tenant setting** in the Power BI/Fabric admin portal (or re-scope it back to `BI Labeling – Enabled Users`) to stop labeling/inheritance/mandatory labeling immediately — settings changes take effect going forward, not retroactively.

• **Turn mandatory labeling off** first if it generates creator friction; keep source/export inheritance on. Keep `Analytics Scope – Map` as the record of the intended state.

Order of operations

`Assess labels + tenant settings → design intent + map → publish labels to pilot creators → enable tenant settings for pilot group (labeling + inheritance + export) → pilot on real content → validate → expand (+ mandatory labeling) → report coverage`

5. Testing Guide — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview)

Run after each configuration step. Every test is safe (read-only or a `-WhatIf` preview) except where it asks you to observe real behavior on labeled content. Validate on `Analytics Labels – Pilot Ring – Stage1` / the pilot workspace before treating the tenant as production-ready.

Test 1 — Prerequisites and enablement are in place

Do: Run `IPPBI-Check-Prereqs.ps1`. **Expected:** ExchangeOnlineManagement + MicrosoftPowerBIMgmt modules present and connected; Information Protection Admin / Power BI Admin roles reported; the sensitivity-label and Power BI Pro/PPU signals reported; whether labels are published is reported. Every gap (e.g. AIP entitlement missing → labeling deferred) is **recorded, not guessed**.

Test 2 — Discovery inventory and tenant-setting checklist are complete

Do: Run `IPPBI-Discover.ps1`; open the Power BI/Fabric admin portal → Tenant settings → Information protection. **Expected:** Every sensitivity label and its publishing policy are listed; the in-scope workspaces are listed; the information-protection tenant-setting checklist is printed and matches the admin portal.

Test 3 — Publish dry-run makes no changes

Do: Run `IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -PolicyMembers "BI-Labeling-Pilot@contoso.com" -WhatIf`. **Expected:** It states the policy/label/membership it *would* set plus the reverse action and changes nothing; a follow-up `Get-LabelPolicy` shows the policy unchanged.

Test 4 — Labels are published and appear in the service

Do: Run `Get-Label | Format-Table DisplayName, Name` and `Get-LabelPolicy`; as a pilot creator, open a report in the Power BI service and check the sensitivity-label picker. **Expected:** The labels exist; `Analytics Labels – Policy` is published to the pilot creators; the label picker appears for a policy member and **not** for a non-member.

Test 5 — Tenant settings are enabled for the pilot group

Do: In the admin portal, confirm each information-protection setting is **On** and scoped to `BI Labeling – Enabled Users`. **Expected:** "Allow users to apply sensitivity labels", source inheritance, and export restriction are On for the group; mandatory labeling is **Off** during the pilot.

Test 6 — Validation gate passes on the pilot

Do: Run `IPPBI-Validate.ps1`. **Expected:** PASS for each scriptable gate — labels exist and are published to the pilot creators — plus the printed tenant-setting gates to confirm in the admin portal. Any FAIL names the specific missing object.

Test 7 — An artifact can be labeled

Do: As a pilot creator, apply `Confidential` to a semantic model and a report in the pilot workspace. **Expected:** The label is applied and visible on the artifact; a consumer sees the sensitivity indicator.

Test 8 — The label inherits from a labeled source

Do: Build/refresh a report over a **labeled** data source (with source inheritance On). **Expected:** The content **inherits** the source's label (or the higher of source/existing), so classification is not lost at the BI boundary. (Verify the exact inheritance behavior against current Microsoft guidance.)

Test 9 — The label inherits to an export

Do: From a labeled report, **Export to Excel** (and to PowerPoint/PDF). **Expected:** The exported file **carries the sensitivity label and its encryption/usage rights** — opening it prompts for/enforces the label's protection. A download no longer strips protection.

Test 10 — Export restriction behaves as designed

Do: As a user without rights to a protected label, attempt to export/consume the protected content. **Expected:** The export/access is **restricted** per the export-protection setting; an authorized user succeeds. (Verify against your tenant's export-restriction configuration.)

Test 11 — Reporting evidence is produced

Do: Run `IPPBI-Report.ps1 -CsvPath .\AnalyticsLabelsReport.csv`. **Expected:** `Analytics Labels – Coverage Report` lists each label, its publication/target coverage, and the in-scope workspaces — usable as the management evidence artifact and re-runnable on a schedule.

Pass/fail summary

All gates green on `Analytics Labels – Pilot Ring – Stage1` — labels published and applied, inheritance from labeled sources and to exports proven on real content, export restriction behaving as designed, and the coverage report produced — before expanding to the estate and enabling mandatory labeling. Any deferred (licensing-gated) capability is recorded explicitly, not silently skipped.

6. Troubleshooting — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview)

Symptoms, likely causes, and fixes for analytics labeling. Stay in **pilot** while diagnosing — reproduce on `Analytics Labels – Pilot Ring – Stage1` / the pilot workspace before touching the wider estate, and prefer disabling a tenant setting over deleting a label.

The sensitivity-label picker doesn't appear in the Power BI service

- **Tenant setting off.** "Allow users to apply sensitivity labels for content" must be **On** and scoped to a group the user is in (`BI Labeling – Enabled Users`). Confirm in the Power BI/Fabric admin portal.
- **Label not published to the user.** Confirm `Analytics Labels – Policy` targets the user (`Get-LabelPolicy`); add them with `IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -Commit`.
- **Licensing.** The user needs the AIP/label entitlement **and** Power BI Pro/PPU (or a capacity). `IPPBI-Check-Prereqs.ps1` reports the signals; missing licensing → labeling **deferred** for that user.
- **Propagation lag.** Tenant-setting and label-policy changes take time to reach the service. Have the user sign out/in and retry before assuming failure.

The label doesn't inherit from a labeled data source

- **Source-inheritance setting off.** "Apply sensitivity labels from data sources to their data in Power BI" must be **On**. Confirm in the admin portal.

- **The source isn't actually labeled.** Inheritance only flows a label that exists on the source. Verify the upstream source carries a label/classification; if it doesn't, rely on artifact labeling and record inheritance as **partial**.
- **Refresh needed.** Inheritance is evaluated on refresh/build. Refresh the semantic model and re-check.
- **Behavior differs by source type.** Which sources propagate labels is Microsoft-defined and evolving — verify the source type is supported for inheritance in current Microsoft guidance.

An export to Excel/PowerPoint/PDF isn't protected

- **Export-protection setting not configured.** The export inherits the label and its encryption only when the export/restriction settings are configured. Re-check the export-protection tenant settings.
- **Label has no encryption.** A label with no protection/encryption produces an export with no protection — that's expected. Confirm the label used on the content actually applies encryption/usage rights (its Purview label configuration).
- **Unsupported export path.** Some export/print paths behave differently; verify the specific export type against current Microsoft documentation before concluding it's a defect.

Mandatory labeling is blocking creators unexpectedly

- **Enabled too early.** Mandatory labeling should be **Off during the pilot** and enabled only after validation, with creators told first. If it was enabled prematurely, turn it **Off** in the admin portal — it's the highest-friction setting and the first to roll back.
- **Labels not published to all creators.** If mandatory labeling is on but a creator can't see any label (see the first section), they're blocked with no way forward. Fix publication/licensing first, then enable mandatory labeling.

`IPPBI-Set-InfoProtection.ps1 -Commit` fails to publish the policy

- **Not connected / wrong session.** Ensure `Connect-IPPSSession` succeeded (`Get-Command Get-Label` resolves). The label cmdlets are in Security & Compliance PowerShell.
- **Insufficient role.** Publishing labels needs Information Protection / Compliance Administrator. Confirm in Purview → Roles & scopes / Permissions.
- **Parameter shape by module version.** Label/membership parameter names vary across ExchangeOnlineManagement versions; verify against your version and update the module if it's far behind.

`IPPBI-Discover.ps1` can't read workspaces

- **Power BI module/role.** Workspace inventory needs the MicrosoftPowerBIMgmt module,

`Connect-PowerBIServiceAccount`, and Power BI admin rights for `-Scope Organization`. Without them the script still reports labels/policies and prints the tenant-setting checklist — the workspace list is optional.

`IPPBI-Validate.ps1` says a label is "not detected on the policy" but the portal shows it

- **Module-version property shape.** Label-to-policy attachment is exposed under different property names

across module versions. Confirm in the portal (Information Protection → Label policies → the policy → **Labels**); if listed there, treat the validator line as informational and note the module version.

When to stop and escalate

- A required capability is unavailable on your licensing/tenant (AIP entitlement, Pro/PPU/capacity, a

tenant setting) — record it **deferred** and route the licensing decision to the owner; do not design around an unavailable feature.

- **Which content is sensitive** is a stakeholder determination (compliance/privacy/data-governance), not

a technical fix — capture the mapping in `Analytics Scope – Map` and defer the determination.

- If inheritance or export protection can't be reconciled with the settings, keep the tenant settings

scoped to the pilot group, gather `IPPBI-Validate.ps1` + `IPPBI-Report.ps1` output as evidence, and raise with Microsoft support rather than expanding a configuration that doesn't behave as designed.

7. Best Practices — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview)

- **Confirm the analytics information-protection surface before you design anything.** The Power BI/Fabric

tenant-setting names, the export-restriction behavior, and the Fabric-item coverage are evolving — read them from the admin portal and Microsoft's current docs for *your* tenant rather than assuming a setting or an entitlement that may have moved.

- **Reuse your existing sensitivity labels — don't invent BI-only labels.** Analytics consumes the same

Purview Information Protection labels as files and email. One taxonomy across documents, mail, and BI is what makes classification consistent and inheritance meaningful.

- **Export inheritance is the point — validate it first.** The reason to label analytics is that the

Excel/PowerPoint/PDF export carries the label and its encryption. Prove that end-to-end on the pilot before anything else; a labeling program that doesn't protect exports has missed its own goal.

- **Turn source inheritance on so classification flows, not restarts.** Where upstream sources are

labeled, let the label inherit into the content so the BI layer isn't a place where protection quietly drops. Where sources are unlabeled, rely on artifact labeling and record inheritance as **partial**.

- **Delegate the tenant settings to a group and pilot there.** Scope the information-protection settings

to `BI Labeling – Enabled Users` and enable for a pilot workspace/group first — never flip them tenant-wide on day one. Mandatory labeling and export restriction are felt immediately by creators and consumers.

- **Turn on mandatory labeling last, and communicate first.** Requiring a label on new/changed content

drives coverage but surprises creators mid-flow if sprung on them. Enable it only after the pilot validates and after you've told creators what will change.

- **Right-size export restriction.** Blocking every export is rarely the intent; the intent is that a

protected export **stays protected**. Configure the restriction so authorized users still get their work done and unauthorized access is what's blocked.

- **Mind the licensing dependency per user and per workspace.** Labeling needs the AIP entitlement **and**

Power BI Pro/PPU (or a capacity) for the affected users. Confirm both; where a user or workspace lacks the licensing, record the capability as **deferred** rather than assuming it applies.

- **Keep automation safe-by-default and secret-free.** Run discovery, validation, and reporting read-only;

gate every write behind `-WhatIf` → `-Commit`; sign in interactively with modern auth. No credentials, tokens, or secrets belong in a script or a document.

- **Re-report on a schedule.** New workspaces and creators appear constantly. Re-run `IPPBI-Report.ps1`

periodically so `Analytics Labels – Coverage Report` stays a live evidence artifact, not a one-time snapshot.

- **Keep determinations with their owners.** The solution configures and evidences analytics labeling;

which content is sensitive, and any regulatory/records obligation on the data it surfaces, is a determination for compliance, privacy, and data-governance. Keep that separation explicit.

8. Deliverables Summary — Sensitivity Labeling for Power BI & Microsoft Fabric (Microsoft Purview)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	Sensitivity labeling for Power BI/Fabric artifacts and their exports, with source and export inheritance; how analytics honors your existing Purview labels (the why)

Document	Purpose
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: assess labels + tenant settings, design intent, publish labels to pilot creators, enable the information-protection tenant settings, pilot on real content, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Sensitivity labels (existing)	`General`, `Confidential`, `Highly Confidential`
Label policy publishing to BI creators	`Analytics Labels – Policy`
Tenant-setting delegation group	`BI Labeling – Enabled Users`
Pilot ring (users) / pilot workspace	`Analytics Labels – Pilot Ring – Stage1`
Scope/design map	`Analytics Scope – Map`
Coverage report	`Analytics Labels – Coverage Report`

PowerShell scripts

Script	What it does
`IPPBI-Discover.ps1`	Reports current state: every sensitivity label and its publishing policy, the in-scope Power BI/Fabric workspaces, and the information-protection tenant-setting checklist to confirm — Security & Compliance PowerShell + read-only Power BI; read-only
`IPPBI-Check-Prereqs.ps1`	Prerequisite/health helper: modules, connections, roles, the label and Power BI Pro/PPU signals, and whether labels are published — PASS/GAP per signal; read-only
`IPPBI-Set-InfoProtection.ps1`	Configuration/Remediation: publishes/points `Analytics Labels – Policy` at the pilot (or expansion) creators and prints the exact Power BI/Fabric tenant-setting checklist to apply; **defaults to** `-WhatIf` (prints the plan + reverse action), applies only with `-Commit`; tenant settings are completed in the admin portal; no secrets in code

Script	What it does
`IPPBI-Validate.ps1`	Validation: PASS/FAIL that the labels exist and are published in `Analytics Labels – Policy` to the intended creators, plus the printed tenant-setting gates to confirm — read-only
`IPPBI-Report.ps1`	Coverage report — each label, its publication/target coverage, and the in-scope workspaces — written to CSV as `Analytics Labels – Coverage Report`; read-only

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Analytics content and its exports protected from Purview — Power BI/Fabric artifacts labeled, labels inherited from labeled sources and carried into Excel/PowerPoint/PDF exports with their encryption — proven on a pilot before you scale, with the determination of which content is sensitive left to the owners who make it.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

IPPBI-Check-Prereqs.ps1

Purpose: Confirms the environment is ready to enable Power BI/Fabric sensitivity labeling: the required PowerShell modules and connections, the least-privilege roles (informational), the sensitivity-label and Power BI Pro/PPU signals, and whether the sensitivity labels are already published. Read-only. HOW TO RUN .\IPPBI-Check-Prereqs.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and (optionally) Connect-PowerBIServiceAccount. No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal. GAPs are recorded against the Requirements checklist and remediated before configuring. Makes no change. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only. Verify your licensing/tenant: AIP entitlement + Power BI Pro/PPU (or capacity) are required; confirm in the M365 admin center. > [CmdletBinding()] param()

How to run: .\IPPBI-Check-Prereqs.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and (optionally) Connect-PowerBIServiceAccount. No secret is stored in the script. EXPECTED RESULTS A PASS/GAP line per signal. GAPs are recorded against the Requirements checklist and remediated before configuring. Makes no change. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only. Verify your licensing/tenant: AIP entitlement + Power BI Pro/PPU (or capacity) are required; confirm in the M365 admin center. > [CmdletBinding()] param()

IPPBI-Discover.ps1

Purpose: Reports the current state for analytics labeling: every sensitivity label and the label policies that publish them, the in-scope Power BI/Fabric workspaces, and a printed checklist of the information-protection tenant settings to confirm in the Power BI/Fabric admin portal (those settings are not fully readable from PowerShell). Read-only — makes no change. HOW TO RUN .\IPPBI-Discover.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and (optionally) Connect-PowerBIServiceAccount for the workspace inventory. No secret is stored in the script. EXPECTED RESULTS A label/policy inventory, a workspace list, and the tenant-setting checklist — captured into 'Analytics Scope - Map'. Makes no configuration change. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only discovery. Verify your licensing/tenant: the information-protection tenant-setting names evolve; confirm them in the admin portal. > [CmdletBinding()] param()

How to run: .\IPPBI-Discover.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession and (optionally) Connect-PowerBIServiceAccount for the workspace inventory. No secret is stored in the script. EXPECTED RESULTS A label/policy inventory, a workspace list, and the tenant-setting checklist — captured into 'Analytics Scope - Map'. Makes no configuration change. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only discovery. Verify your licensing/tenant: the information-protection tenant-setting names evolve; confirm them in the admin portal. > [CmdletBinding()] param()

IPPBI-Report.ps1

Purpose: Builds 'Analytics Labels - Coverage Report' — for each sensitivity label: its display name, the label policy that publishes it, and the policy's target coverage — plus a count of in-scope Power BI/Fabric

workspaces (when the Power BI module is connected) — as the management evidence artifact. Read-only; writing a CSV is opt-in. HOW TO RUN .\IPPBI-Report.ps1 .\IPPBI-Report.ps1 -CsvPath .\AnalyticsLabelsReport.csv PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (and optionally Connect-PowerBIServiceAccount). No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) of each label, its publishing policy, and target coverage — re-runnable on a schedule. Makes no configuration change. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only reporting. Verify your licensing/tenant: policy label-attachment and target property names vary by module version. > [CmdletBinding()] param([string[]]$Labels = @('General','Confidential','Highly Confidential'), [string]$CsvPath)

How to run: .\IPPBI-Report.ps1 .\IPPBI-Report.ps1 -CsvPath .\AnalyticsLabelsReport.csv PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (and optionally Connect-PowerBIServiceAccount). No secret is stored in the script. EXPECTED RESULTS A table (and optional CSV) of each label, its publishing policy, and target coverage — re-runnable on a schedule. Makes no configuration change. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only reporting. Verify your licensing/tenant: policy label-attachment and target property names vary by module version. > [CmdletBinding()] param([string[]]$Labels = @('General','Confidential','Highly Confidential'), [string]$CsvPath)

IPPBI-Set-InfoProtection.ps1

Purpose: Automates the safe part of enabling analytics labeling: publishes / points 'Analytics Labels - Policy' at the pilot (or expansion) BI creators. DEFAULTS TO -WhatIf: prints exactly what it would do plus the reverse action and changes nothing; applies only with -Commit. The information- protection TENANT SETTINGS (labeling, source inheritance, export restriction, mandatory labeling) are configured in the Power BI/Fabric admin portal — this script prints the exact checklist for -Action ShowTenantSettings but does not change them. No credentials or secrets are stored. HOW TO RUN .\IPPBI-Set-InfoProtection.ps1 -Action ShowTenantSettings .\IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -PolicyMembers "BI-Labeling-Pilot@contoso.com" -WhatIf .\IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -PolicyMembers "BI-Labeling-Pilot@contoso.com" -Commit PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (modern-auth/MFA admin). EXPECTED RESULTS ShowTenantSettings: the admin-portal checklist to apply. PublishPolicy -WhatIf: the plan + reverse command, no change. PublishPolicy -Commit: the policy exists/targets the members afterward (verify with Get-LabelPolicy). .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Safe-by-default. Verify your licensing/tenant: tenant-setting names evolve; complete them in the admin portal. Pilot before enforcing. > [CmdletBinding()] param([ValidateSet('ShowTenantSettings','PublishPolicy')] [string]$Action = 'ShowTenantSettings', [string]$PolicyName = 'Analytics Labels - Policy', [string[]]$Labels = @('General','Confidential','Highly Confidential'), [string[]]$PolicyMembers = @(), [string]$DelegationGroup = 'BI Labeling - Enabled Users', [switch]$WhatIf, [switch]$Commit)

How to run: .\IPPBI-Set-InfoProtection.ps1 -Action ShowTenantSettings .\IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -PolicyMembers "BI-Labeling-Pilot@contoso.com" -WhatIf .\IPPBI-Set-InfoProtection.ps1 -Action PublishPolicy -PolicyMembers "BI-Labeling-Pilot@contoso.com" -Commit PowerShell 5.1+/7+. Connect first with Connect-IPPSSession (modern-auth/MFA admin). EXPECTED RESULTS ShowTenantSettings: the admin-portal checklist to apply. PublishPolicy -WhatIf: the plan + reverse command, no change. PublishPolicy -Commit: the policy exists/targets the members afterward (verify with Get-LabelPolicy). .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Safe-by-default. Verify your licensing/tenant: tenant-setting names evolve; complete them in the admin portal. Pilot before enforcing. > [CmdletBinding()] param(

[ValidateSet('ShowTenantSettings','PublishPolicy')] [string]$Action = 'ShowTenantSettings', [string]$PolicyName = 'Analytics Labels - Policy', [string[]]$Labels = @('General','Confidential','Highly Confidential'), [string[]]$PolicyMembers = @(), [string]$DelegationGroup = 'BI Labeling - Enabled Users', [switch]$WhatIf, [switch]$Commit)

IPPBI-Validate.ps1

Purpose: The pilot/expansion gate. Confirms PASS/FAIL that each intended sensitivity label exists and is published in 'Analytics Labels - Policy' to the intended creators, then prints the information- protection tenant-setting gates to confirm in the Power BI/Fabric admin portal (not readable here). Read-only — makes no change. Run on the pilot before expanding, and after each expansion wave. HOW TO RUN .\IPPBI-Validate.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per scriptable gate plus the printed tenant-setting gates; a final overall PASS only when every intended label exists and is published. Any FAIL names the specific missing object. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only gate. Verify your licensing/tenant: the tenant settings are confirmed in the admin portal; policy property names vary by module version. > [CmdletBinding()] param([string]$PolicyName = 'Analytics Labels - Policy', [string[]]$ExpectedLabels = @('General','Confidential','Highly Confidential'))

How to run: .\IPPBI-Validate.ps1 PowerShell 5.1+/7+. Connect first with Connect-IPPSSession. No secret is stored in the script. EXPECTED RESULTS A PASS/FAIL line per scriptable gate plus the printed tenant-setting gates; a final overall PASS only when every intended label exists and is published. Any FAIL names the specific missing object. .NOTES Purview — Sensitivity Labeling for Power BI & Fabric. Read-only gate. Verify your licensing/tenant: the tenant settings are confirmed in the admin portal; policy property names vary by module version. > [CmdletBinding()] param([string]$PolicyName = 'Analytics Labels - Policy', [string[]]$ExpectedLabels = @('General','Confidential','Highly Confidential'))